

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐA PHƯƠNG TIỆN



BÁO CÁO

Môn Học: Phát triển ứng dụng Iot

Đề Tài : Xây dựng hệ thống Iot cho phòng thông minh

Giảng viên	: Nguyễn Quốc Uy
Họ và tên sinh viên	: Điều Chính Hiếu – B22DCPT087
Nhóm lớp	: 02

Hà Nội – 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN

1.1 Giới thiệu đề tài

1.1.1. Bối cảnh chung và sự phát triển của IoT

Trong kỷ nguyên công nghiệp 4.0, Internet of Things (IoT — Vạn vật kết nối) đã chuyển mình từ một khái niệm lý thuyết thành một xu hướng công nghệ mang tính cốt lõi. Sự phát triển mạnh mẽ của cơ sở hạ tầng mạng và các thiết bị phần cứng nhỏ gọn đã cho phép hàng tỷ thiết bị trên toàn cầu kết nối, thu thập và trao đổi dữ liệu. Từ các hệ thống nhà thông minh, nông nghiệp công nghệ cao cho đến tự động hóa công nghiệp, IoT đang từng bước tối ưu hóa chất lượng cuộc sống và nâng cao hiệu suất vận hành.

Đặc biệt trong lĩnh vực nông nghiệp, ứng dụng IoT (còn gọi là *Smart Farming* hay *Precision Agriculture*) mang lại lợi ích to lớn: giảm lãng phí tài nguyên, tối ưu điều kiện môi trường canh tác, và cho phép người nông dân giám sát — điều khiển từ xa mà không cần hiện diện tại chỗ.

1.1.2. Giới thiệu về hệ thống xây dựng

Nhận thức được tầm quan trọng đó, đề tài này tập trung vào việc nghiên cứu và phát triển một "**Hệ thống IoT Giám sát và Điều khiển Nông trại Thông minh**" — *Humble Dieu Smart Farm*.

Hệ thống sử dụng vi điều khiển **ESP8266 (NodeMCU)** kết hợp với các cảm biến để liên tục đo lường và thu thập ba thông số môi trường cốt lõi: **Nhiệt độ**, **Độ ẩm** và **Cường độ ánh sáng**.

Điểm nổi bật của hệ thống là việc ứng dụng giao thức truyền bản tin nhẹ **MQTT** để đảm bảo khả năng truyền tải dữ liệu realtime với độ trễ thấp. Toàn bộ dữ liệu được gửi về hệ thống máy chủ backend vận hành trên nền tảng **NestJS (TypeScript)**, sau đó được xử lý và lưu trữ vào cơ sở dữ liệu quan hệ **PostgreSQL**.

Người dùng có thể theo dõi trực quan các biến động môi trường và trực tiếp ra lệnh điều khiển các thiết bị ngoại vi (đèn LED) thông qua **giao diện web React** cập nhật thời gian thực qua **Socket.IO**.

1.2. Lý do chọn đề tài

Việc thực hiện đề tài này xuất phát từ các lý do chính sau:

1. **Tính ứng dụng thực tế cao:** Nhu cầu giám sát môi trường sống và điều khiển thiết bị từ xa đang ngày càng trở nên thiết thực. Mô hình của hệ thống có thể mở rộng để áp dụng vào quản lý vi khí hậu trong nhà màng, cảnh báo cháy nổ, hoặc tự động hóa thiết bị điện trong nông trại.

2. **Bắt kịp xu hướng công nghệ:** Đề tài áp dụng các công nghệ đang được ưa chuộng trong ngành: kiến trúc module NestJS, MQTT Broker, WebSocket realtime và React SPA.

3. **Phát triển kỹ năng toàn diện:** Quá trình xây dựng đòi hỏi sự kết hợp giữa lập trình phần cứng (Arduino C++), phát triển backend (NestJS/TypeScript), thiết kế CSDL (PostgreSQL) và xây dựng giao diện (React/Tailwind CSS).

1.3. Mục tiêu đề tài

1.3.1. Mục tiêu tổng quát

Mục tiêu cốt lõi là nghiên cứu, thiết kế và xây dựng thành công một hệ thống IoT hoàn chỉnh từ tầng thiết bị vật lý (phần cứng) đến tầng ứng dụng (phần mềm). Hệ thống phải đảm bảo thu thập dữ liệu môi trường chính xác, truyền tải theo thời gian thực ổn định, và cho phép người dùng giám sát — điều khiển thiết bị từ xa qua nền tảng mạng Internet.

1.3.2. Các mục tiêu cụ thể

1. Về mặt Phần cứng

- Thiết kế sơ đồ nguyên lý, lắp ráp và lập trình thành công vi điều khiển **ESP8266 (NodeMCU)**.
- Tích hợp cảm biến **DHT11** (nhiệt độ + độ ẩm) và **LDR** (cường độ ánh sáng) để đọc liên tục mỗi 2 giây.
- Xây dựng mạch điều khiển **3 đèn LED** (LED_TEMP_01, LED_HUM_01, LED_LDR_01) đóng vai trò thiết bị actuator.

2. Về mặt Kết nối

- Cấu hình ESP8266 kết nối mạng WiFi ổn định.
- Thiết lập giao thức **MQTT (QoS 1)** để đảm bảo Publish dữ liệu cảm biến và Subscribe lệnh điều khiển giữa thiết bị và Broker với độ trễ thấp nhất.
- Xây dựng cơ chế **tự kết nối lại** (auto-reconnect) và khôi phục trạng thái GPIO từ CSDL.

3. Về mặt Phần mềm Máy chủ (Backend) & Cơ sở dữ liệu

- Xây dựng backend với **NestJS (TypeScript)** theo kiến trúc module, Dependency Injection.
- Thiết kế CSDL **PostgreSQL** tối ưu lưu trữ: thông tin cảm biến, lịch sử đo đạc và nhật ký điều khiển.
- Phát triển **RESTful API** đầy đủ với Swagger documentation tại `/api/docs`.
- Tích hợp **WebSocket / Socket.IO** để đẩy dữ liệu realtime tới client.
- Xây dựng **hệ thống logging** với Winston (ghi `app.log` và `error.log`).

4. Về mặt Giao diện Người dùng (Web App)

- Xây dựng giao diện **Dashboard** trực quan, hiển thị ba thẻ số liệu thời gian thực và biểu đồ AreaChart sliding window.

- Tích hợp **bảng điều khiển thiết bị** với toggle ON/OFF, xác nhận trạng thái qua MQTT ACK.
- Cung cấp trang **lịch sử dữ liệu cảm biến** và **nhật ký hoạt động** với bộ lọc và phân trang.
- Mọi cập nhật thực hiện qua **Socket.IO** không cần tải lại trang.

1.4. Phạm vi hệ thống

Hệ thống được thiết kế và xây dựng dưới dạng mô hình thử nghiệm (prototype), nhằm chứng minh tính khả thi của việc kết hợp các công nghệ IoT hiện đại. Phạm vi cụ thể được giới hạn như sau:

1.4.1. Phạm vi phần cứng

- **Vi điều khiển:** Hệ thống sử dụng **ESP8266 (NodeMCU)** làm node mạng trung tâm do có sẵn WiFi tích hợp, phù hợp cho truyền tải dữ liệu IoT.
- **Cảm biến:** Thu thập ba thông số môi trường cơ bản — Nhiệt độ, Độ ẩm và Cường độ ánh sáng.
- **Thiết bị actuator:** Sử dụng 3 đèn LED tượng trưng cho các thiết bị nông trại (quạt thông gió, máy bơm, đèn chiếu sáng).

1.4.2. Phạm vi phần mềm và Mạng

- **Backend & Cơ sở dữ liệu:** NestJS (TypeScript) + PostgreSQL, lưu trữ các bảng cốt lõi: thông tin cảm biến, lịch sử đo đạc và nhật ký điều khiển.
- **Giao thức:** MQTT cho truyền nhận dữ liệu realtime giữa ESP8266 và Backend; HTTP REST + WebSocket/Socket.IO cho giao tiếp giữa Backend và Web App.

1.4.3. Phạm vi chức năng

- Giám sát thông số Nhiệt độ, Độ ẩm, Ánh sáng và cập nhật lên giao diện liên tục.

- Điều khiển bật/tắt thiết bị từ xa với xác nhận trạng thái hai chiều.
- Tra cứu lịch sử dữ liệu cảm biến và nhật ký lệnh điều khiển.

1.4.4. Ngoài phạm vi:

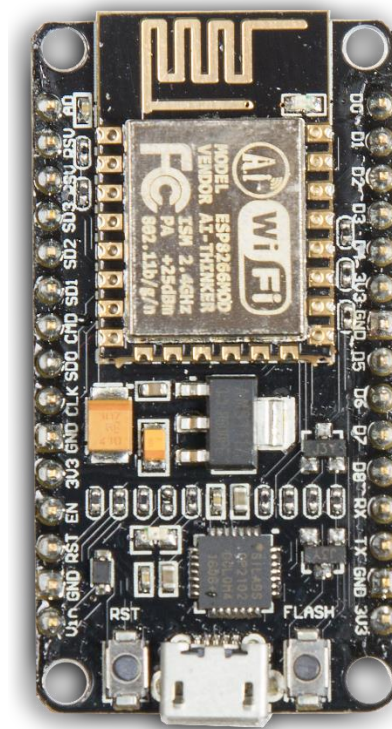
- Xác thực người dùng / phân quyền, hỗ trợ đa nông trại, ứng dụng di động native.

1.5. Công nghệ sử dụng

Hệ thống được phát triển dựa trên sự kết hợp của nhiều công nghệ hiện đại, bao phủ từ phần cứng vi mạch đến phần mềm máy chủ và giao diện người dùng.

1. Nền tảng Phần cứng

- **ESP8266 (NodeMCU):** Vi điều khiển chi phí thấp, tích hợp sẵn WiFi 802.11 b/g/n, phổ biến trong hệ sinh thái Arduino IoT.



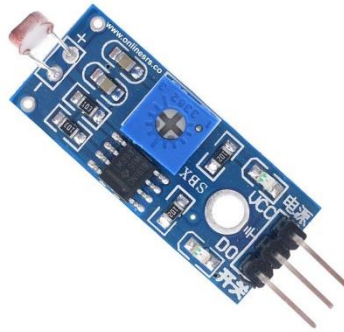
Hình 1.1: Vi điều khiển ESP8266 NodeMCU

- **Cảm biến DHT11:** Cảm biến số đo nhiệt độ (0–50°C, $\pm 2^\circ\text{C}$) và độ ẩm (20–90% RH, $\pm 5\%$), giao tiếp single-wire.

- **LDR (Light Dependent Resistor):** Cảm biến quang trở đọc cường độ ánh sáng (0–1023 Lux) qua cổng Analog A0.



Hình 1.2: Module cảm biến DHT11

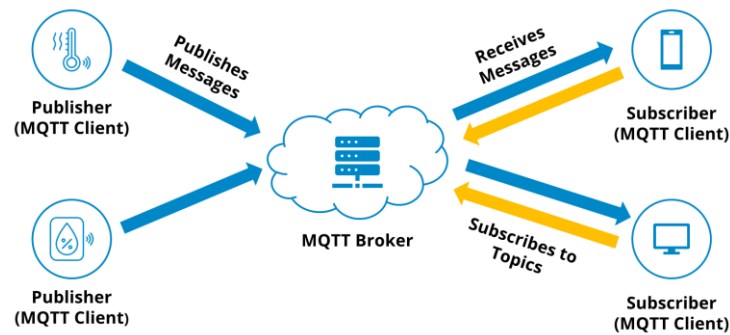


Hình 1.3: Module cảm biến LDR

- **Đèn LED:** Thiết bị actuator mô phỏng — 3 LED kết nối GPIO D5 (GPIO 14), D6 (GPIO 12), D7 (GPIO 13).

2. Giao thức

- **MQTT:** Giao thức truyền bản tin theo mô hình Publish/Subscribe, được thiết kế đặc biệt cho thiết bị IoT bằng thông thấp. Hệ thống sử dụng **Mosquitto Broker** với QoS 1, đảm bảo mỗi tin nhắn được giao ít nhất một lần.



Hình 1.4: Mô hình giao thức MQTT Pub/Sub

- **RESTful API:** Giao thức tiêu chuẩn HTTP cho giao tiếp Client–Server. Backend cung cấp các endpoint truy xuất lịch sử và gửi lệnh điều khiển. Tài liệu tự động sinh tại `/api/docs` (Swagger/OpenAPI 3).



Hình 1.5: Swagger UI tài liệu REST API

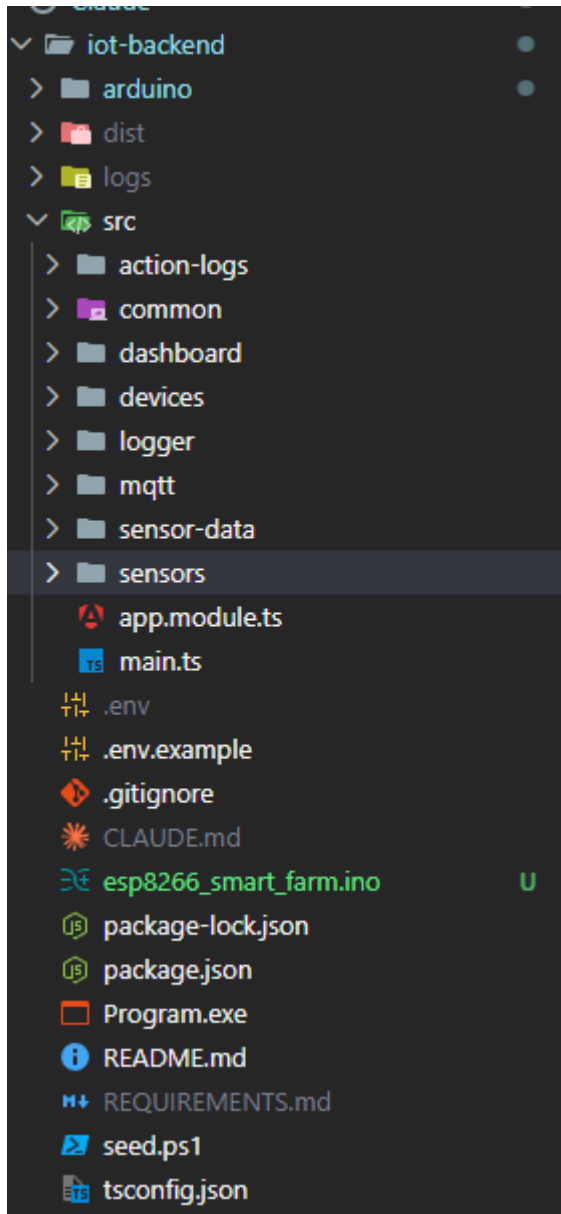
- **Socket.IO (WebSocket):** Công nghệ giao tiếp hai chiều liên tục, cho phép Backend chủ động đẩy (push) dữ liệu realtime xuống giao diện người dùng mà không cần polling.



Hình 1.6: Socket.IO

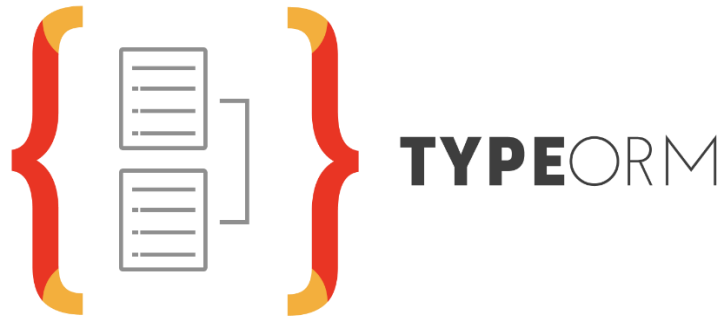
3. Nền tảng Backend và Cơ sở dữ liệu

- **NestJS v10:** Framework TypeScript cho Node.js, áp dụng kiến trúc module + Dependency Injection, tương tự Spring Boot. Hỗ trợ decorator-based, dễ mở rộng và bảo trì.



Hình 1.6: Kiến trúc module NestJS

- **TypeORM:** ORM cho TypeScript/Node.js, tự động đồng bộ schema từ entity (auto-sync). Hỗ trợ query builder và migration.



Hình 1.8: TypeORM

- **PostgreSQL:** Hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, lưu trữ toàn bộ dữ liệu hệ thống.



Hình 1.7: PostgreSQL

- **Winston:** Logger cấu trúc, ghi log ra file `app.log` và `error.log`.

4. Nền tảng Web App (Frontend)

- **React 18 + Vite:** Single Page Application với Hot Module Replacement, hiệu suất phát triển cao.
- **Tailwind CSS v3:** Utility-first CSS framework, xây dựng UI đáp ứng nhanh chóng.
- **Recharts:** Thư viện biểu đồ SVG cho React — AreaChart với gradient màu sắc cho từng loại cảm biến.
- **Socket.IO Client:** Hai kết nối song song tới namespace `/sensors` và `/devices`.
- **Axios:** HTTP client với interceptor xử lý lỗi tự động.



Hình 1.8: Dashboard giao diện web React

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1. Kiến trúc tổng thể

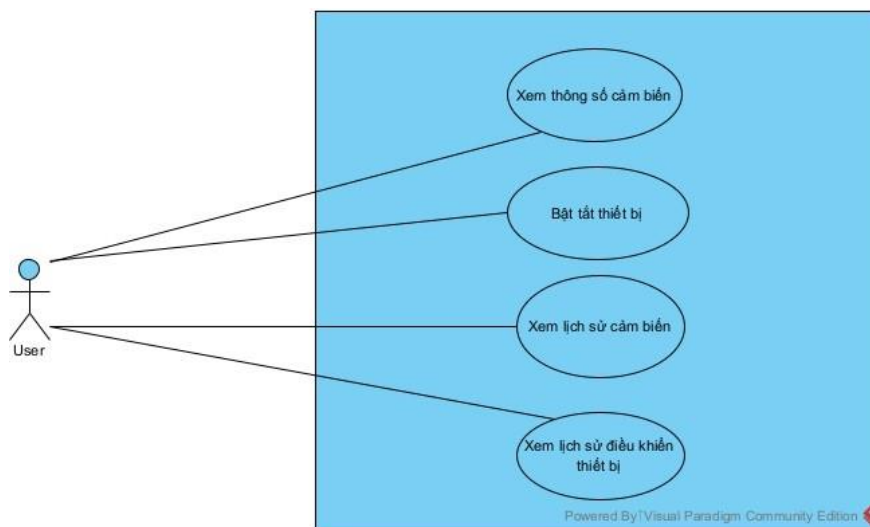
Hệ thống IoT được thiết kế theo **mô hình kiến trúc phân lớp ba tầng** chuẩn IoT, đảm bảo tính module hóa cao, dễ bảo trì, nâng cấp và mở rộng. Kiến trúc hệ thống được chia thành 4 lớp chính:

Lớp	Thành phần	Vai trò
Edge (Phần cứng)	ESP8266 + DHT11 + LDR + 3 LED	Đọc cảm biến, điều khiển thiết bị, giao tiếp MQTT
Broker	Mosquitto MQTT Broker	Môi giới tin nhắn — định tuyến dữ liệu và lệnh
Platform (Backend)	NestJS + PostgreSQL	Lưu trữ, REST API, WebSocket realtime
Application (Frontend)	React 18 + Vite + Tailwind	Dashboard, điều khiển, bảng dữ liệu lịch sử

Giao thức giao tiếp:

- MQTT (cổng 2411): ESP8266 ↔ Mosquitto Broker ↔ Backend
- HTTP/REST (cổng 3000, tiền tố `/api/v1`): Frontend ↔ Backend
- WebSocket / Socket.IO (cổng 3000): Backend → Frontend (realtime push)

2.2 Phân tích Use Case



Hình 2.2: Biểu đồ Use Case hệ thống

1. Xác định Tác nhân (Actor)

- **Người dùng (User):** Là người trực tiếp vận hành hệ thống thông qua giao diện web. Tác nhân này có quyền theo dõi dữ liệu môi trường theo thời gian thực và ra lệnh điều khiển các thiết bị ngoại vi từ xa.
- **ESP8266 (Node IoT):** Thiết bị phần cứng đóng vai trò tác nhân ngoại vi — liên tục đọc cảm biến, phản hồi lệnh điều khiển và gửi ACK.
- **MQTT Broker (Mosquitto):** Trung gian định tuyến tin nhắn giữa ESP8266 và Backend.

2. Các trường hợp sử dụng chính (Use Cases)

Use Case	Tác nhân	Mô tả
Xem thông số cảm biến realtime	Người dùng	Theo dõi Nhiệt độ, Độ ẩm, Ánh sáng cập nhật liên tục trên Dashboard
Xem biểu đồ lịch sử	Người dùng	Quan sát xu hướng dữ liệu qua AreaChart sliding window 20 điểm
Bật/Tắt thiết bị	Người dùng	Gửi lệnh điều khiển LED từ giao diện web, chờ xác nhận ACK
Xem lịch sử dữ liệu cảm biến	Người dùng	Tra cứu bảng dữ liệu lịch sử với bộ lọc và phân trang
Xem nhật ký điều khiển	Người dùng	Kiểm tra lại các lần thao tác với trạng thái PROCESSING / SUCCESS / FAILURE
Gửi dữ liệu cảm biến	ESP8266	Publish lên topic <code>sensor/data</code> mỗi 2 giây
Nhận và thực thi lệnh	ESP8266	Subscribe <code>system/control</code> , bật/tắt LED, gửi ACK lên <code>system/state</code>

Use Case	Tác nhân	Mô tả
Kết nối lại và khôi phục	ESP8266	Sau mất điện, publish <code>device/init/request</code> để nhận lại trạng thái từ CSDL

2.3 Biểu đồ tuần tự

Sơ đồ 1: Lấy dữ liệu cảm biến hiển thị lên Web App



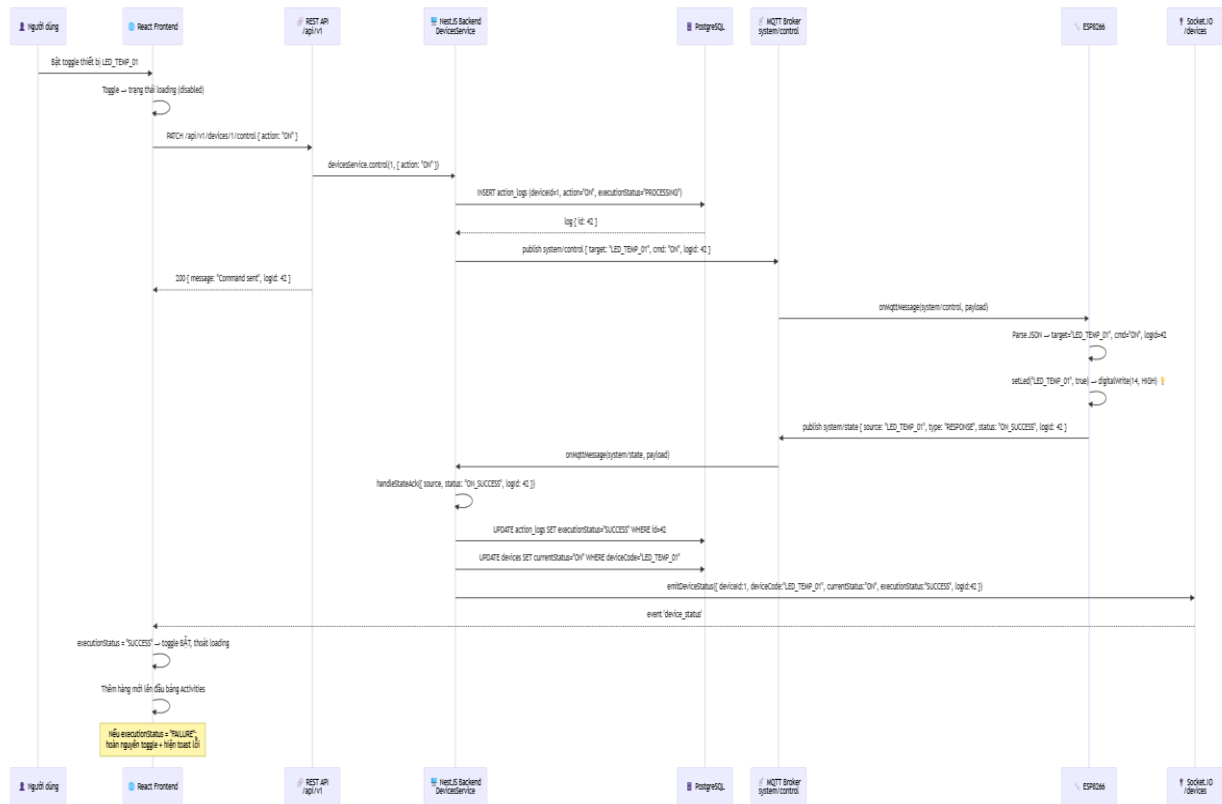
Hình 2.3: Sơ đồ tuần tự thu thập và hiển thị dữ liệu cảm biến

Mô tả các bước:

1. ESP8266 đọc DHT11 mỗi 2000ms → lấy temp=28.5°C, hum=65.0%; đọc LDR → light=512 Lux.
2. Publish 3 message riêng biệt lên topic `sensor/data` với payload { `sensorCode`, `value` } (QoS 1).
3. Mosquitto Broker định tuyến từng message đến `SensorsService` (đã subscribe từ `onModuleInit`).

4. SensorsService parse JSON, tìm sensor trong DB theo sensorCode và isActive=true.
5. Cập nhật lastSeen = NOW() cho sensor, sau đó INSERT bản ghi mới vào sensor_data với status="Normal".
6. Gọi SensorsGateway.emitSensorData() → Socket.IO phát event sensor_data đến tất cả client trên namespace /sensors.
7. React Dashboard nhận event: cập nhật thẻ Hero theo type, đẩy điểm mới vào chart (xóa điểm cũ nếu vượt 20), thêm hàng lên đầu bảng Sensors.

Sơ đồ 2: Bật/Tắt thiết bị



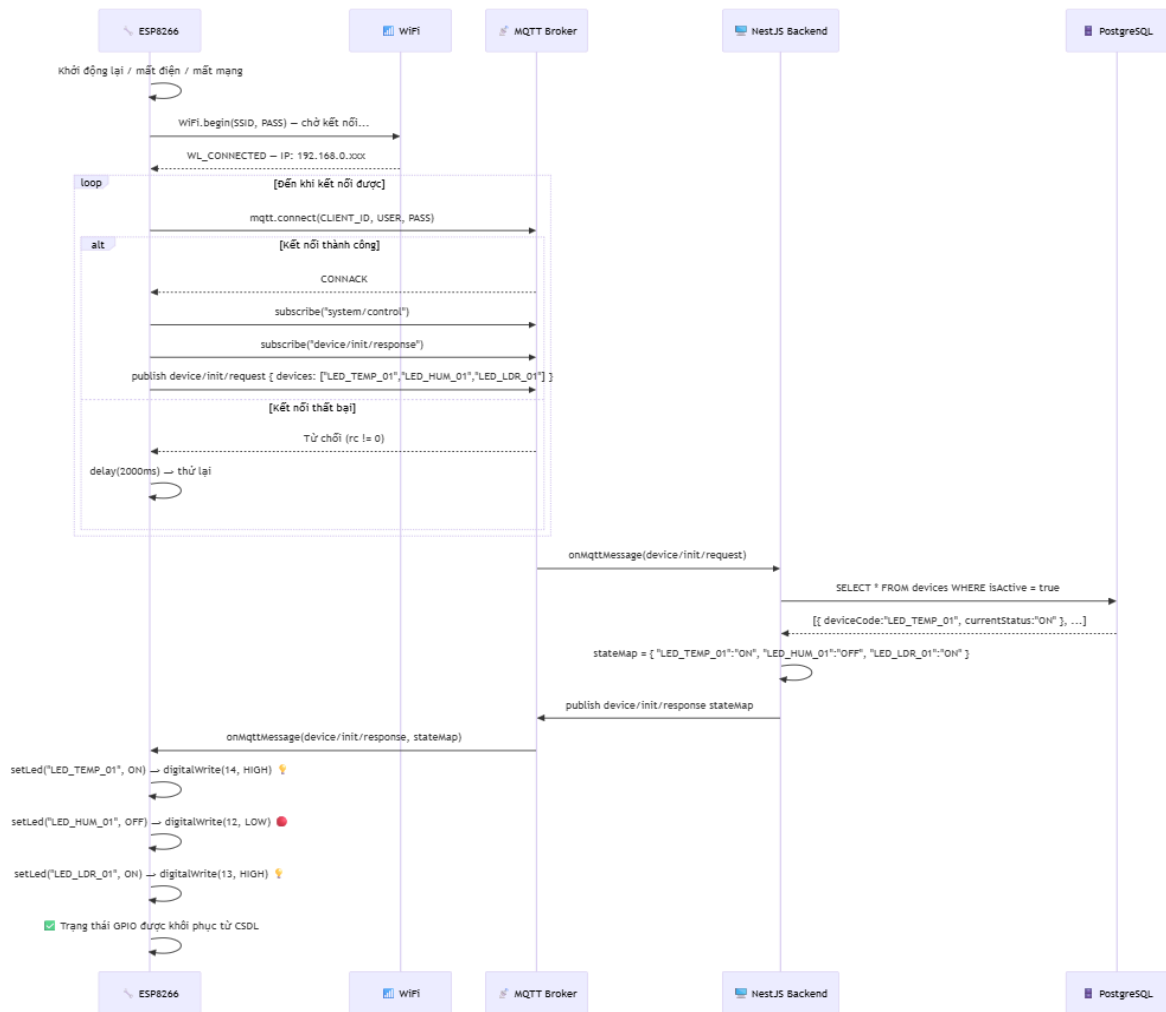
Hình 2.4: Sơ đồ tuần tự điều khiển thiết bị

Mô tả các bước:

1. Người dùng bật toggle LED_TEMP_01 trên Dashboard → React đặt toggle vào trạng thái loading (disabled).
2. Gọi PATCH /api/v1/devices/1/control { action: "ON" }.
3. DeviceService.control() tạo bản ghi action_logs với executionStatus="PROCESSING" → nhận lại logId=42.

4. Publish lên `system/control: { target:"LED_TEMP_01", cmd:"ON", logId:42 }` (QoS 1).
5. API trả về 200 `{ message:"Command sent", logId:42 }` ngay lập tức — không chờ ACK từ phần cứng.
6. ESP8266 nhận message từ Broker → parse → `setLed("LED_TEMP_01", true)` → `digitalWrite(14, HIGH)`.
7. ESP8266 publish ACK lên `system/state: { source:"LED_TEMP_01", type:"RESPONSE", status:"ON_SUCCESS", logId:42 }`.
8. `DevicesService` nhận ACK: `isSuccess = status.endsWith("_SUCCESS")` → true → UPDATE `action_logs` thành SUCCESS, UPDATE `devices.currentStatus = "ON"`.
9. `DevicesGateway.emitDeviceStatus()` → `Socket.IO` phát `device_status` đến tất cả client trên namespace `/devices`.
10. React nhận event: `executionStatus="SUCCESS"` → toggle chuyển sang ON, thoát loading; thêm hàng mới lên đầu bảng Activities. Nếu FAILURE → revert toggle + hiện Toast lỗi 4 giây.

Sơ đồ 3: Kết nối lại và khôi phục trạng thái ESP8266



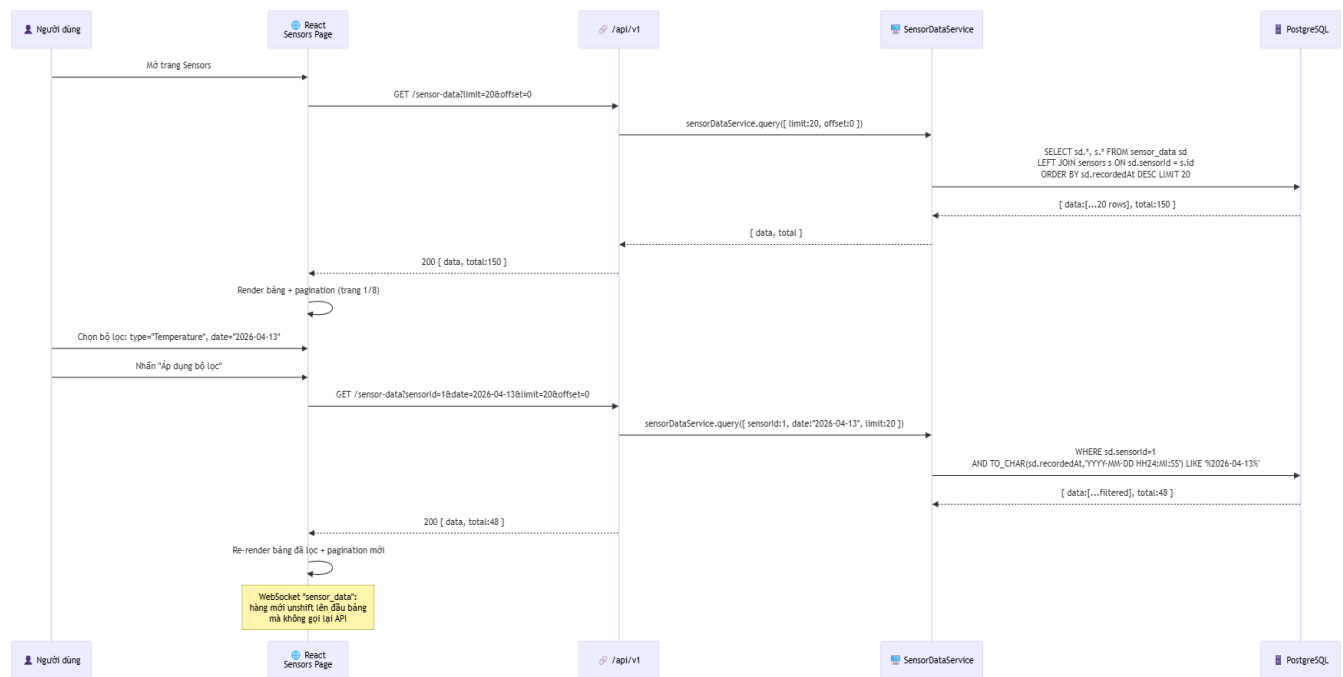
Hình 2.5: Sơ đồ tuần tự kết nối lại sau mất nguồn

Mô tả các bước:

1. ESP8266 khởi động lại (mất điện / reset) → gọi `WiFi.begin(SSID, PASS)`, chờ `WL_CONNECTED`.
2. Gọi `mqtt.connect()` — lặp lại mỗi 2 giây cho đến khi nhận `CONNACK` từ Broker.
3. Sau khi kết nối thành công: subscribe `system/control` và `device/init/response`.

4. Publish `device/init/request`: {
`devices: ["LED_TEMP_01", "LED_HUM_01", "LED_LDR_01"]` }.
5. `DevicesService` nhận request → query DB: `SELECT deviceCode, currentStatus FROM devices WHERE isActive=true`.
6. Build `stateMap` và publish lên `device/init/response`: {
`"LED_TEMP_01": "ON", "LED_HUM_01": "OFF",`
`"LED_LDR_01": "ON"` }.
7. ESP8266 nhận response → duyệt từng entry trong `stateMap` → gọi `setLed(deviceCode, status=="ON")` → `digitalWrite()` tương ứng.
8. GPIO được khôi phục hoàn toàn từ CSDL, không phụ thuộc bộ nhớ cục bộ của vi điều khiển.

Sơ đồ 4: Hiện thị Lịch sử Dữ liệu Cảm biến



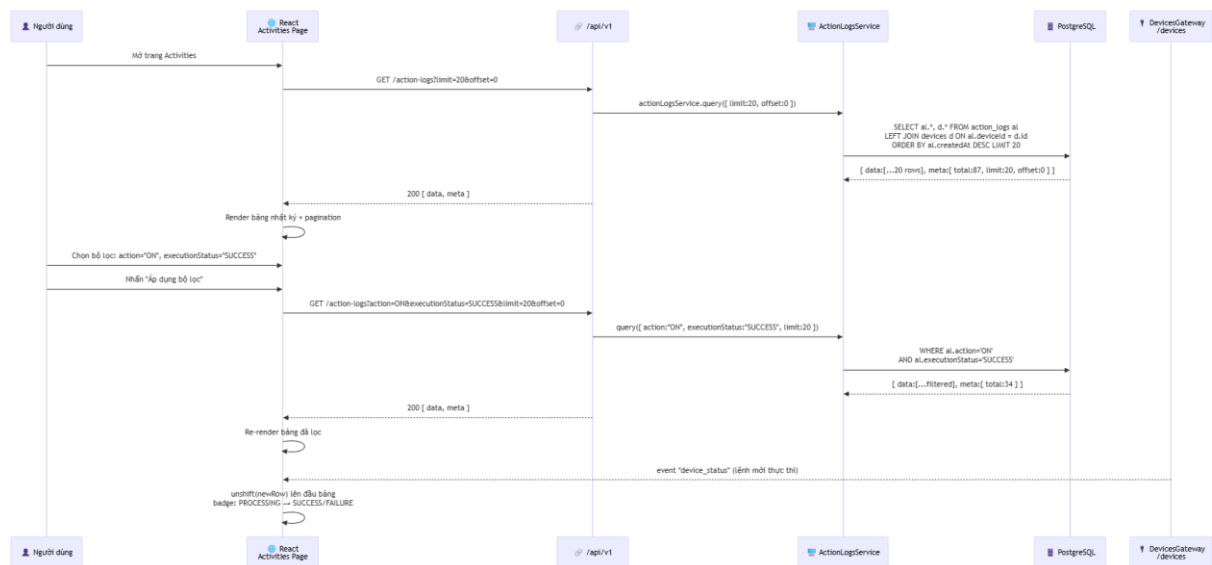
Hình 2.6: Sơ đồ tuần tự tra cứu lịch sử dữ liệu cảm biến

Mô tả các bước:

1. Người dùng mở trang Sensors → React gọi `GET /api/v1/sensor-data?limit=20&offset=0`.

2. `SensorDataService.query()` dùng `QueryBuilder`: `LEFT JOIN sensors, ORDER BY recordedAt DESC, TAKE 20`.
3. DB trả về 20 bản ghi mới nhất kèm `total=150` → render bảng phân trang
4. Người dùng chọn bộ lọc loại cảm biến và ngày, nhấn "Áp dụng".
5. Gọi lại API với `sensorId` và `date` — backend lọc bằng `TO_CHAR(recordedAt, 'YYYY-MM-DD HH24:MI:SS') LIKE '%2026-04-13%'`.
6. Kết quả lọc được render lại với pagination mới theo `total` trả về.
7. Trong suốt quá trình, WebSocket `sensor_data` vẫn hoạt động: mỗi bản ghi mới được `unshift` lên đầu bảng mà không cần gọi lại API.

Sơ đồ 5: Hiện thị Nhật ký Hoạt động



Hình 2.7: Sơ đồ tuần tự tra cứu nhật ký điều khiển

Mô tả các bước:

1. Người dùng mở trang Activities → React gọi `GET /api/v1/action-logs?limit=20&offset=0`.
2. `ActionLogsService.query()` dùng `QueryBuilder`: `LEFT JOIN devices, ORDER BY createdAt DESC, TAKE 20`.
3. DB trả về 20 bản ghi mới nhất (kèm thông tin thiết bị) và `meta.total` → render bảng phân trang.

4. Người dùng chọn bộ lọc đa tiêu chí: `action`, `executionStatus`, `deviceId`, `date` — nhấn "Áp dụng".
5. API được gọi lại với các tham số lọc; backend áp dụng từng `andWhere()` có điều kiện vào `QueryBuilder`.
6. Kết quả lọc được render lại với `meta.total` mới.
7. WebSocket `device_status` (từ `DevicesGateway`) tự động thêm hàng mới lên đầu bảng khi có lệnh điều khiển mới — hàng hiển thị badge `PROCESSING` cho đến khi nhận ACK từ ESP8266 cập nhật thành `SUCCESS` hoặc `FAILURE`.

CHƯƠNG 3: XÂY DỰNG HỆ THỐNG

3.1 Thiết kế Cơ sở dữ liệu

1. Mục đích thiết kế

Cơ sở dữ liệu của hệ thống được thiết kế nhằm đáp ứng các yêu cầu cốt lõi:

- Lưu trữ thông tin định danh các loại cảm biến và dữ liệu đo đạc thực tế theo từng mốc thời gian.
- Lưu trữ thông tin của các thiết bị điều khiển ngoại vi (LED đại diện cho quạt, bơm, đèn).
- Ghi nhận chi tiết nhật ký (lịch sử) các hành động bật/tắt thiết bị với vòng đời trạng thái đầy đủ.
- Đảm bảo cấu trúc tối ưu để truy xuất nhanh chóng khi vẽ biểu đồ, tra cứu lịch sử và đồng bộ trạng thái.

3.2. Cấu trúc các bảng dữ liệu chính

3.2.1. Bảng sensors

Mô tả: Lưu trữ thông tin danh mục các cảm biến đang hoạt động trong hệ thống. Quan hệ 1–N với bảng `sensor_data`.

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	integer	PK, tự tăng	Khóa chính
<code>name</code>	varchar	NOT NULL	Tên hiển thị, ví dụ: "Temp Sensor 01"
<code>sensorCode</code>	varchar	UNIQUE, NOT NULL	Mã máy, ví dụ: <code>DTH_TEMP_01</code>
<code>type</code>	varchar	NOT NULL	"Temperature" "Humidity" "Light"
<code>unit</code>	varchar	NULLABLE	"°C" "%" "Lux"
<code>isActive</code>	boolean	DEFAULT <code>true</code>	Cờ xóa mềm (soft-delete)
<code>lastSeen</code>	timestampz	NULLABLE	Thời điểm nhận MQTT cuối cùng
<code>createdAt</code>	timestampz	AUTO	Thời điểm tạo bản ghi

3.2.2. Bảng sensor_data

Mô tả: Lưu trữ chuỗi dữ liệu thực tế đo được từ cảm biến theo từng mốc thời gian. Phục vụ vẽ biểu đồ và tra cứu lịch sử.

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	integer	PK, tự tăng	Khóa chính
<code>sensorId</code>	integer	FK → <code>sensors(id)</code> CASCADE	Tham chiếu cảm biến
<code>value</code>	float	NOT NULL	Giá trị đo (ví dụ: 28.5)
<code>status</code>	varchar	DEFAULT "Normal"	"Normal" "Warning"
<code>recordedAt</code>	timestampz	AUTO	Thời điểm ghi nhận dữ liệu

3.2.3. Bảng `devices`

Mô tả: Lưu trữ thông tin danh mục các thiết bị ngoại vi. Quan hệ 1–N với bảng `action_logs`.

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	integer	PK, tự tăng	Khóa chính
<code>name</code>	varchar	NOT NULL	Tên hiển thị, ví dụ: "Ventilation Fan"
<code>deviceCode</code>	varchar	UNIQUE, NOT NULL	Mã máy, ví dụ: <code>LED_TEMP_01</code>
<code>type</code>	varchar	NOT NULL	Loại thiết bị
<code>currentStatus</code>	varchar	DEFAULT "OFF"	"ON" "OFF" — trạng thái đã xác nhận
<code>isActive</code>	boolean	DEFAULT <code>true</code>	Cờ xóa mềm

Cột	Kiểu	Ràng buộc	Mô tả
<code>createdAt</code>	timestampz	AUTO	Thời điểm tạo bản ghi

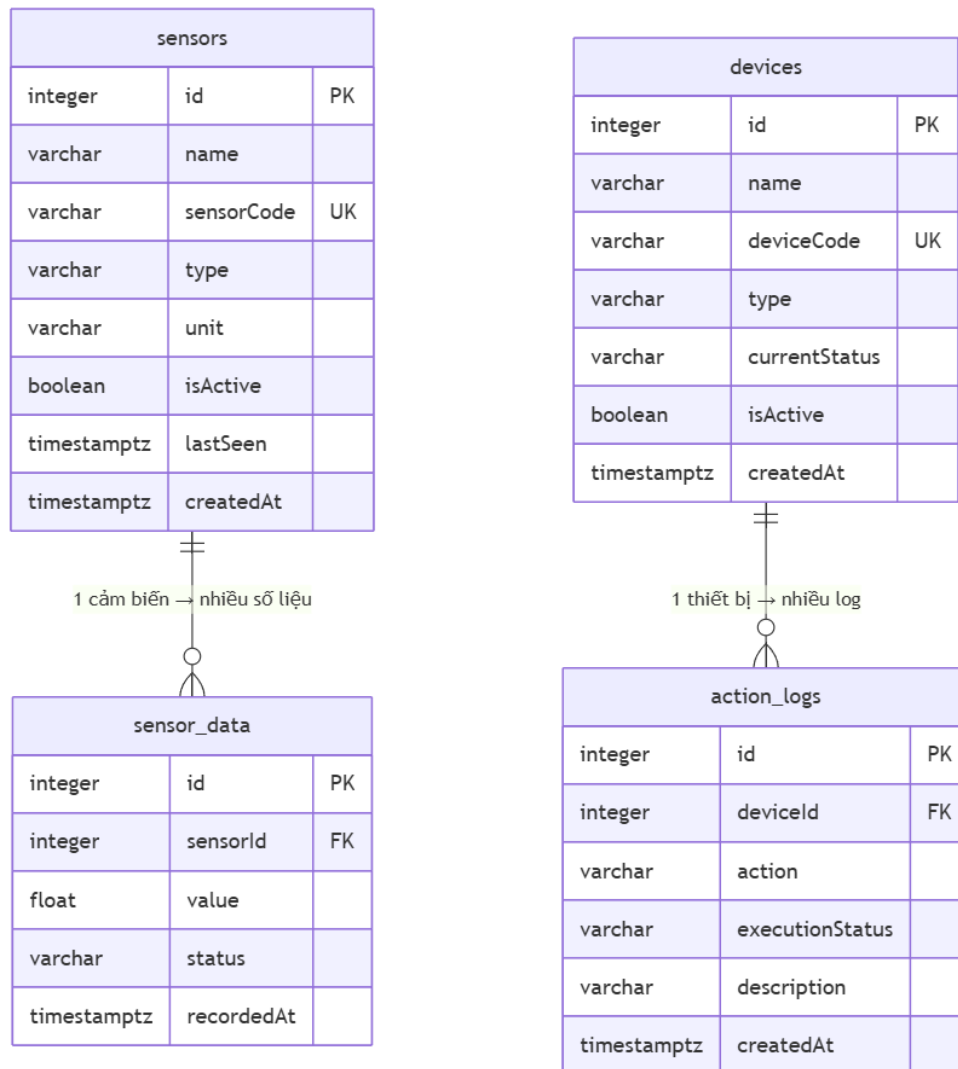
3.2.4. Bảng `action_logs`

Mô tả: Nhật ký kiểm tra toàn diện mọi lệnh điều khiển thiết bị. Cho phép khôi phục trạng thái khi ESP8266 khởi động lại.

Cột	Kiểu	Ràng buộc	Mô tả
<code>id</code>	integer	PK, tự tăng	Khóa chính
<code>deviceId</code>	integer	FK → <code>devices(id)</code> CASCADE	Tham chiếu thiết bị
<code>action</code>	varchar	NOT NULL	"ON" "OFF"
<code>executionStatus</code>	varchar	DEFAULT "PROCESSING"	"PROCESSING" "SUCCESS" "FAILURE"
<code>description</code>	varchar	NULLABLE	Ghi chú hoặc thông báo lỗi từ ESP8266

Cột	Kiểu	Ràng buộc	Mô tả
<code>createdAt</code>	timestampz	AUTO	Thời điểm gửi lệnh

3.2.4. Lược đồ quan hệ ERD



Hình 3.1: Lược đồ quan hệ cơ sở dữ liệu ERD

3.3. Xây dựng Backend

3.3.1. Thiết kế các RESTful API

Tất cả endpoint có tiền tố `/api/v1`. Swagger UI tự động sinh tại `/api/docs`.

Nhóm API Quản lý và Điều khiển Thiết bị (`/devices``)

Nhóm này chịu trách nhiệm khởi tạo danh sách thiết bị ngoại vi và xử lý luồng điều khiển trạng thái (ON/OFF) thông qua MQTT.

Phương thức	Đường dẫn	Body / Query	Phản hồi
GET	/devices	—	Danh sách thiết bị đang hoạt động
GET	/devices/:id	—	Thiết bị theo ID
POST	/devices	{ name, deviceCode, type }	Thiết bị đã tạo (201)
PATCH	/devices/:id/ control	{ action: "ON" "OFF" }	{ message: "Command sent", logId }
DELETE	/devices/:id	—	204 No Content (soft- delete)

Nhóm API Quản lý Cảm biến (`/sensors`)

Phương thức	Đường dẫn	Query	Phản hồi
GET	/sensors	?page&limit&search	{ data: [...], meta: { page, limit, total, totalPages } }

Phương thức	Đường dẫn	Query	Phản hồi
GET	/sensors/ :id	—	Cảm biến theo ID
POST	/sensors	{ name, sensorCode, type, unit }	Cảm biến đã tạo
DELETE	/sensors/ :id	—	204 No Content

Nhóm API Dữ liệu Cảm biến (`/sensor-data`)

Phương thức	Đường dẫn	Query	Phản hồi
GET	/sensor-data	?sensorId&date& limit&offset	{ data: [...], total }
GET	/sensor- data/sensor/:sens orId/latest	—	Bản ghi mới nhất của cảm biến

Nhóm API Nhật ký Hoạt động (`/action-logs`)

Phương thức	Đường dẫn	Query	Phản hồi
GET	/action-logs	?deviceId&action&ex ecutionStatus&date& limit&offset	{ data: [...], meta }

Phương thức	Đường dẫn	Query	Phản hồi
GET	/action-logs/device/:deviceId	?limit&offset	Log theo thiết bị cụ thể

Nhóm API Dashboard (`/dashboard`)

Phương thức	Đường dẫn	Query	Phản hồi
GET	/dashboard/charts	?limit (tối đa 100)	{ datasets: [...] } — dữ liệu vẽ biểu đồ
GET	/dashboard/latest	—	{ readings: [...] } — số liệu mới nhất
GET	/dashboard/devices	—	{ devices: [...] } — trạng thái thiết bị

Hình 3.2: Giao diện Swagger UI tài liệu REST API

3.3.2. Cấu hình và Quản lý giao thức MQTT

Cấu hình kết nối MQTT Broker

Backend kết nối tới Mosquitto Broker với các thông số cấu hình từ file `.env`:

```
You, 2 weeks ago | 1 author (You)
# MQTT Broker Configuration
MQTT_HOST=localhost
MQTT_PORT=1883
MQTT_USERNAME=your_username
MQTT_PASSWORD=your_password
MQTT_CLIENT_ID=iot-backend
```

Hình 3.2. Cấu hình MQTT trong file `.env`

Các tùy chọn kết nối: `reconnectPeriod: 5000ms`, `connectTimeout: 10000ms`, `clean: true`.

3.3.3. Thiết lập Topic MQTT

Hệ thống định nghĩa 5 topic chuyên dụng để phân tách rõ ràng các luồng nghiệp vụ:

Topic	Chiều	Định dạng Payload	Mục đích
<code>sensor/data</code>	ESP8266 → Backend	<code>{ sensorCode, value }</code>	Gửi số liệu cảm biến mỗi 2 giây
<code>system/control</code>	Backend → ESP8266	<code>{ target, cmd, logId }</code>	Lệnh BẬT/TẮT thiết bị
<code>system/state</code>	ESP8266 → Backend	<code>{ source, type, status, logId }</code>	ACK sau khi thay đổi trạng thái LED
<code>device/init/request</code>	ESP8266 → Backend	<code>{ devices: [...] }</code>	Yêu cầu trạng thái thiết bị khi kết nối lại
<code>device/init/response</code>	Backend → ESP8266	<code>{ LED_TEMP_01: "OFF", ... }</code>	Phản hồi trạng thái từ CSDL

Mức QoS: Tất cả tin nhắn dùng QoS 1 (giao tối thiểu một lần).

Wildcard: MqttService hỗ trợ `+` (một cấp) và `#` (nhiều cấp).

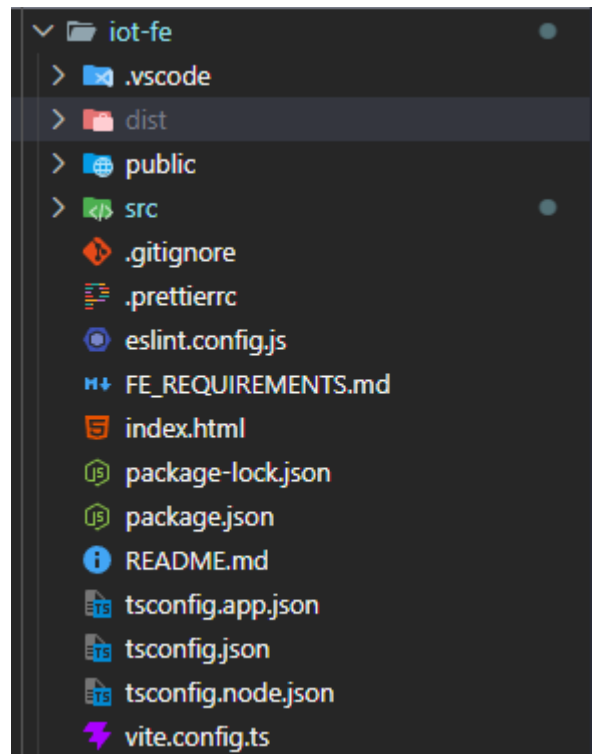
Auto-reconnect: Sau ngắt kết nối, backend tự kết nối lại và đăng ký lại tất cả topic.

WebSocket Gateway

Gateway	Namespace	Sự kiện	Payload	Kích hoạt khi
Sensor sGateway	/sensors	sensor_data	{ sensorCode, type, unit, value, status, recordedAt }	Mỗi lần MQTT được lưu vào DB
Device sGateway	/devices	device_status	{ deviceId, deviceCode, currentStatus, executionStatus , logId }	Nhận ACK từ ESP8266 trên system/sta te

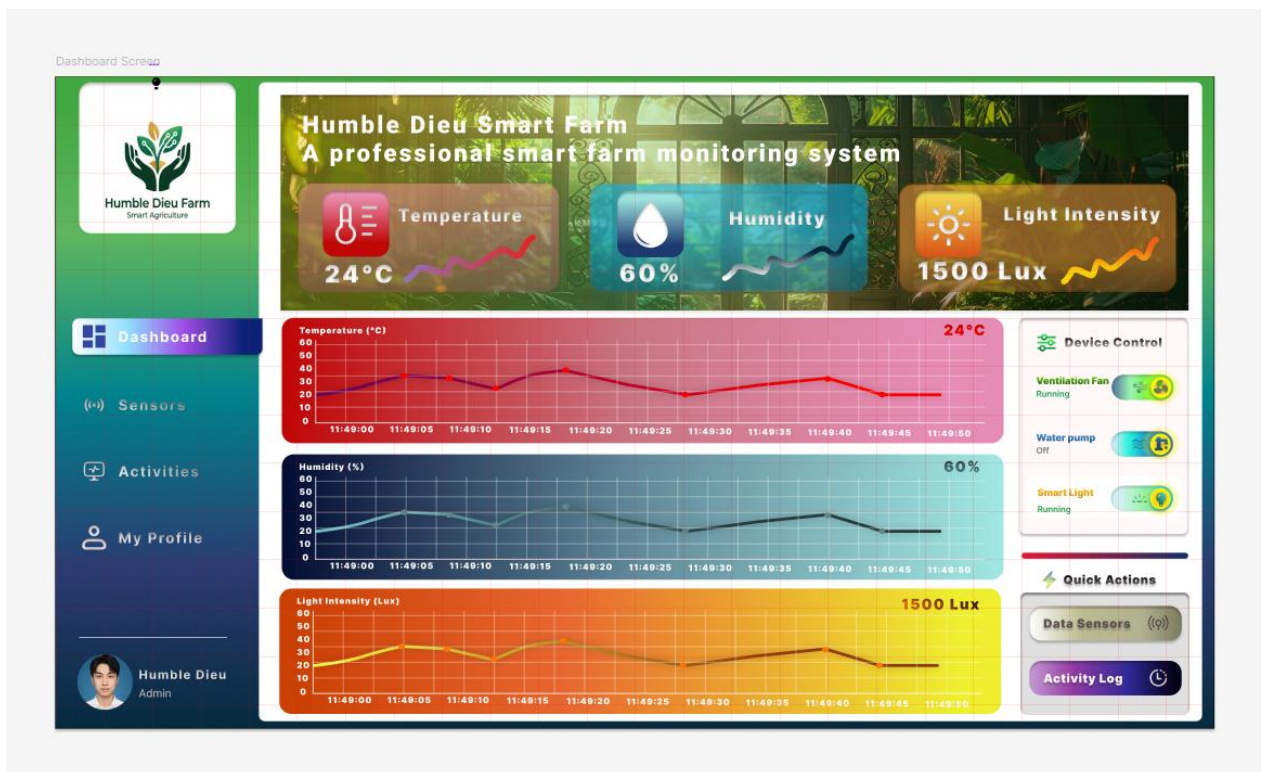
3.4. Xây dựng Frontend

Ứng dụng web được thiết kế với tông màu tối chủ đạo kết hợp gradient màu sắc nổi bật, mang lại trải nghiệm thị giác hiện đại. Cấu trúc ứng dụng được chia thành **4 trang chính**, điều hướng qua Sidebar cố định bên trái.



Hình 3.3. Cấu trúc thư mục Frontend

3.4.1. Trang Dashboard



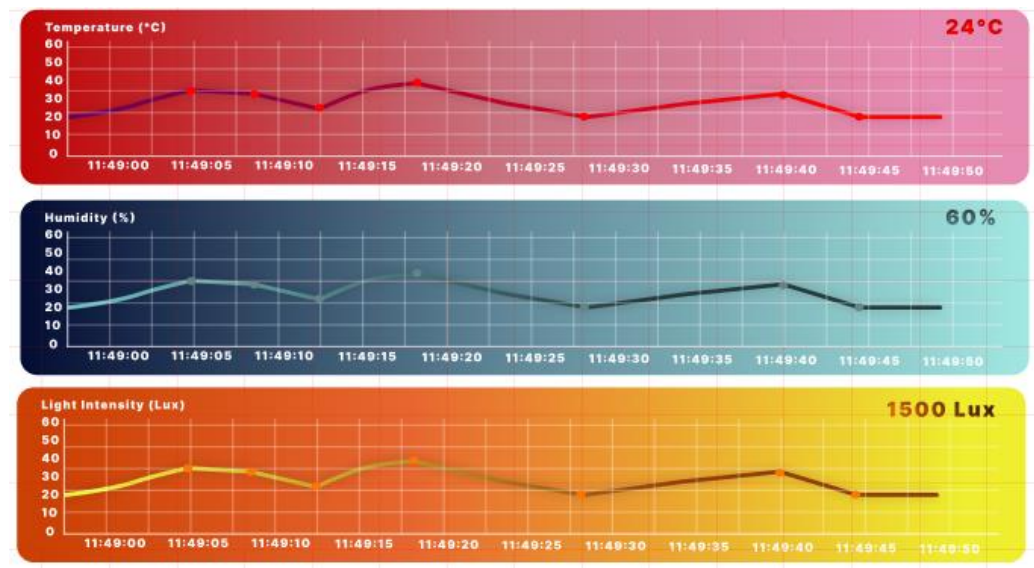
Hình 3.4: Giao diện Dashboard — thẻ thông số và biểu đồ realtime

Khu vực Hero Banner: Hiện thị ảnh nền nông trại với lớp phủ tối. Ba thẻ số liệu thời gian thực:

Thẻ	Biểu tượng	Màu nền	Nguồn dữ liệu
Nhiệt độ	Nhiệt kế	Đỏ	GET /dashboard/latest → type = Temperature
Độ ẩm	Giọt nước	Xanh lam	GET /dashboard/latest → type = Humidity
Cường độ ánh sáng	Mặt trời	Vàng	GET /dashboard/latest → type = Light

Realtime: Cập nhật ngay khi nhận sự kiện `sensor_data` từ WebSocket khớp theo `type`.

Khu vực Biểu đồ Cảm biến (SensorCharts): Ba `AreaChart` `Recharts` xếp chồng, mỗi biểu đồ tải 20 số liệu cuối khi mount và duy trì **cửa sổ trượt 20 điểm** qua WebSocket. Vùng tô smooth monotone với SVG gradient: Nhiệt độ (đỏ), Độ ẩm (xanh ngọc), Ánh sáng (vàng).



Hình 3.5: Biểu đồ AreaChart sliding window realtime

Khu vực Điều khiển Thiết bị (SensorControl):

Thiết bị	Biểu tượng	Màu gradient
Ventilation Fan (LED_TEMP_01)	Quạt	Xanh lá
Smart Pump (LED_HUM_01)	Bơm nước	Xanh lam
Smart Light (LED_LDR_01)	Bóng đèn	Vàng

Luồng toggle (không cập nhật trước khi nhận ACK):

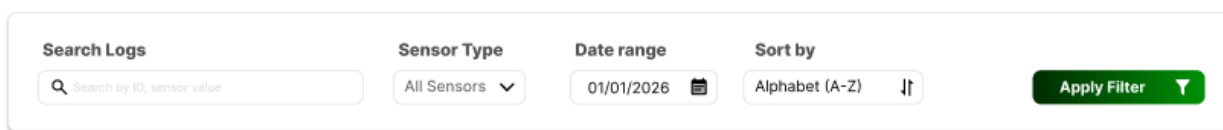
1. Người dùng bật/tắt → gọi PATCH `/devices/:id/control`
2. Toggle vào trạng thái loading (disabled) ngay lập tức
3. Chờ sự kiện `device_status` từ WebSocket
4. SUCCESS → cập nhật toggle thành ON/OFF

5. FAILURE → hoàn nguyên toggle + hiển thị toast lỗi



Hình 3.6: Bảng điều khiển thiết bị với toggle và trạng thái loading

3.4.2. Trang Dữ liệu Cảm biến



Hình 3.7: Bộ lọc thông minh trang Sensor Data

Bộ lọc thông minh: Tìm kiếm tên, loại cảm biến, ngày, thứ tự sắp xếp. Chỉ áp dụng khi nhấn nút "Áp dụng bộ lọc".

Bảng dữ liệu:

Cột	Nguồn	Hiển thị
SENSOR_ID	sensor.id	#8825
TIMESTAMP	recordedAt	2026-03-28 14:08:12
SENSOR TYPE	sensor.type	Biểu tượng + nhãn có màu
VALUE	value + unit	28.5 °C

SENSOR_ID 	TIMESTAMP 	SENSOR TYPE 	VALUE 
#8825	2025-05-20 14:30:00	 Temperature	25.5 °C
#8826	2025-05-20 14:30:05	 Humidity	86.2%
#8827	2025-05-20 14:30:40	 Light Intensity	1500 Lux
#8826	2025-05-20 14:30:30	 Humidity	70.6%
#8826	2025-05-20 14:31:25	 Humidity	55.5 %
#8827	2025-05-20 14:29:10	 Light Intensity	1420 Lux
#8825	2025-05-20 14:30:44	 Temperature	26.5 °C

Showing 1 to 6 of 1,240 entries








Hình 3.8: Bảng dữ liệu lịch sử cảm biến với phân trang

Realtime: Hàng mới được thêm lên đầu khi nhận `sensor_data` WebSocket.

3.4.3. Trang Nhật ký Hoạt động

Search Logs

Date period (From - To)

Time

Sensor Type

All Sensors

Sort by

Alphabet (A-Z)

Hình 3.9: Bộ lọc thông minh trang Action Logs

Bộ lọc thông minh: Tìm kiếm tên, phạm vi ngày (from/to), thời gian chính xác, loại thiết bị, sắp xếp.

Bảng nhật ký:

Cột	Nguồn	Hiển thị
DEVICE_ID	device.id	#1
TIMESTAMP	createdAt	2026-03-28 14:08:12
DEVICE NAME	device.name	Biểu tượng + tên
ACTION	action + executionStatus	Văn bản màu (TURNED ON / TURNING ON)
ACTION_STATUS	executionStatus	Badge pill

Badge trạng thái ACTION_STATUS:

executionStatus	Text	Style
SUCCESS	✓ Thành công	Nền xanh lá, chữ trắng, bo tròn
FAILURE	✗ Thất bại	Nền đỏ, chữ trắng, bo tròn
PROCESSING	⌚ Đang xử lý	Nền cam, chữ trắng, bo tròn

DEVICE_ID 	TIMESTAMP 	DEVICE NAME 	ACTION 	ACTION_STATUS 
#8825	2025-05-20 14:30:00	 Ventilation Fan	TURNED ON	 Success
#8826	2025-05-20 14:30:05	 Smart Pump	TURNED OFF	 Failed
#8827	2025-05-20 14:30:40	 Smart Light	TURNING OFF	 Pending
#8826	2025-05-20 14:30:30	 Smart Pump	TURNED OFF	 Success
#8826	2025-05-20 14:31:25	 Smart Pump	TURNED ON	 Failed
#8827	2025-05-20 14:29:10	 Smart Light	TURNED OFF	 Failed
#8825	2025-05-20 14:30:44	 Ventilation Fan	TURNED ON	 Success

Hình 3.9: Bảng nhật ký hoạt động với badge trạng thái

Realtime: Hàng mới được thêm lên đầu khi nhận `device_status` WebSocket.

Xử lý lỗi toàn cục

`SocketContext.tsx` duy trì hai kết nối Socket.IO liên tục khi ứng dụng mount:

- `sensorSocket` → namespace `/sensors` → sự kiện `sensor_data`
- `deviceSocket` → namespace `/devices` → sự kiện `device_status`

Axios interceptor bắt mọi 4xx/5xx → phát sự kiện `app:toast` → Toast hiển thị góc trên phải, tự đóng sau 4 giây.

3.5 Phần cứng và Firmware (ESP8266)

3.5.1 Sơ đồ kết nối phần cứng

Linh kiện	Loại	Chân	Mã thiết bị	Chức năng
DHT11	Cảm biến số	D2 (GPIO 4)	—	Đọc nhiệt độ và độ ẩm
LDR	Cảm biến analog	A0 (Analog)	—	Đọc cường độ ánh sáng (0–1023 Lux)
LED_TEM P	Actuator	D5 (GPIO 14)	LED_TEMP_01	Đèn LED — đại diện quạt thông gió
LED_HUM	Actuator	D6 (GPIO 12)	LED_HUM_01	Đèn LED — đại diện máy bơm
LED_LDR	Actuator	D7 (GPIO 13)	LED_LDR_01	Đèn LED — đại diện đèn chiếu sáng

Thư viện Arduino cần thiết:

- ESP8266WiFi — Kết nối WiFi
- PubSubClient (Nick O'Leary) — MQTT client
- DHT sensor library (Adafruit) — Driver DHT11
- ArduinoJson (Benoit Blanchon) — JSON serialize/deserialize

3.5.2. Kiến trúc firmware

1. `setup()` — Giai đoạn khởi tạo:

- Khởi tạo GPIO (LED output LOW, LDR input)
- Khởi động DHT11

- Kết nối WiFi (blocking — chờ đến khi có IP)

- Cấu hình MQTT server, port, callback

2. `loop()` — Vòng lặp chính (non-blocking):

- Kiểm tra trạng thái MQTT → gọi `mqttReconnect()` nếu mất kết nối

- Gọi `mqtt.loop()` để poll tin nhắn đến

- Mỗi **2000ms**: đọc DHT11 + LDR → publish 3 tin nhắn cảm biến riêng biệt

3. `onMqttMessage()` — Xử lý tin nhắn đến:

- `device/init/response`: Phân tích bản đồ trạng thái → khôi phục chân LED

- `system/control`: Phân tích `{ target, cmd, logId }` → gọi `setLed()` → gọi `publishAck()`

4. `mqttReconnect()` — Cơ chế kết nối lại:

- Đăng ký lại: `system/control, device/init/response`

- Publish `device/init/request` → kích hoạt backend gửi trạng thái CSDL

3.5.3. Luồng điều khiển đầy đủ (7 bước)

Bước	Chủ thể	Hành động	Topic / Giao thức
1	Người dùng	Click toggle → <code>PATCH</code> <code>/devices/1/control</code> <code>{ action: "ON" }</code>	HTTP REST
2	Backend	Tạo <code>ActionLog</code> (<code>PROCESSING</code>) → publish lệnh	<code>system/control</code> → <code>{ target, cmd, logId }</code>

Bước	Chủ thể	Hành động	Topic / Giao thức
3	ESP8266	Nhận lệnh → <code>digitalWrite(14, HIGH)</code>	MQTT subscribe
4	ESP8266	Publish ACK xác nhận	<code>system/state → { source, status: "ON_SUCCESS", logId }</code>
5	Backend	Cập nhật ActionLog → <code>SUCCESS, currentStatus = "ON"</code>	PostgreSQL UPDATE
6	Backend	Phát WebSocket tới tất cả client	<code>Socket.IO /devices → device_status</code>
7	Trình duyệt	Nhận <code>device_status</code> → toggle sang BẬT	Socket.IO event

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

4.1 Kết quả đạt được

Sau quá trình nghiên cứu, lắp ráp phần cứng và phát triển phần mềm, hệ thống Smart Farm IoT đã hoàn thiện và đáp ứng được các mục tiêu đề ra ban đầu. Cụ thể:

1. Khả năng hoạt động của phần cứng và luồng truyền dữ liệu

- **Vi điều khiển ESP8266** hoạt động bền bỉ, duy trì kết nối WiFi và MQTT ổn

định trong thời gian dài. Cơ chế auto-reconnect đảm bảo thiết bị tự khôi phục sau sự cố mạng.

- **Các cảm biến** (DHT11 + LDR) đo lường chính xác ba thông số: Nhiệt độ, Độ ẩm và Cường độ ánh sáng. Ba đèn LED đóng/ngắt đúng theo lệnh điều khiển nhận được.
- **Luồng dữ liệu** vận hành trơn tru theo kiến trúc đã thiết kế: ESP8266 publish MQTT → NestJS Backend tiếp nhận, xử lý, lưu PostgreSQL → phân phối đến Web App qua Socket.IO mà không xảy ra mất mát gói tin.
- **Khôi phục trạng thái GPIO** sau mất điện: ESP8266 gửi `device/init/request`, Backend truy vấn CSDL và phản hồi `device/init/response` với trạng thái cuối của từng thiết bị.

2. Độ trễ và hiệu năng realtime

- **Luồng cảm biến:** Ứng dụng giao thức MQTT mang lại hiệu năng xuất sắc. Độ trễ từ lúc ESP8266 đọc cảm biến đến khi biểu đồ trên trình duyệt cập nhật điểm mới chỉ rơi vào khoảng **dưới 1 giây**.
- **Luồng điều khiển:** Lệnh điều khiển truyền xuống phần cứng và phản hồi trạng thái ngược lại trình duyệt gần như tức thời qua chuỗi MQTT ACK → WebSocket. Trạng thái `PROCESSING` chỉ tồn tại trong khoảng thời gian rất ngắn.
- **Nhật ký kiểm tra:** 100% lệnh điều khiển được ghi lại với vòng đời đầy đủ `PROCESSING → SUCCESS / FAILURE`, đảm bảo tính minh bạch và truy vết.

3. Chất lượng API và hệ thống

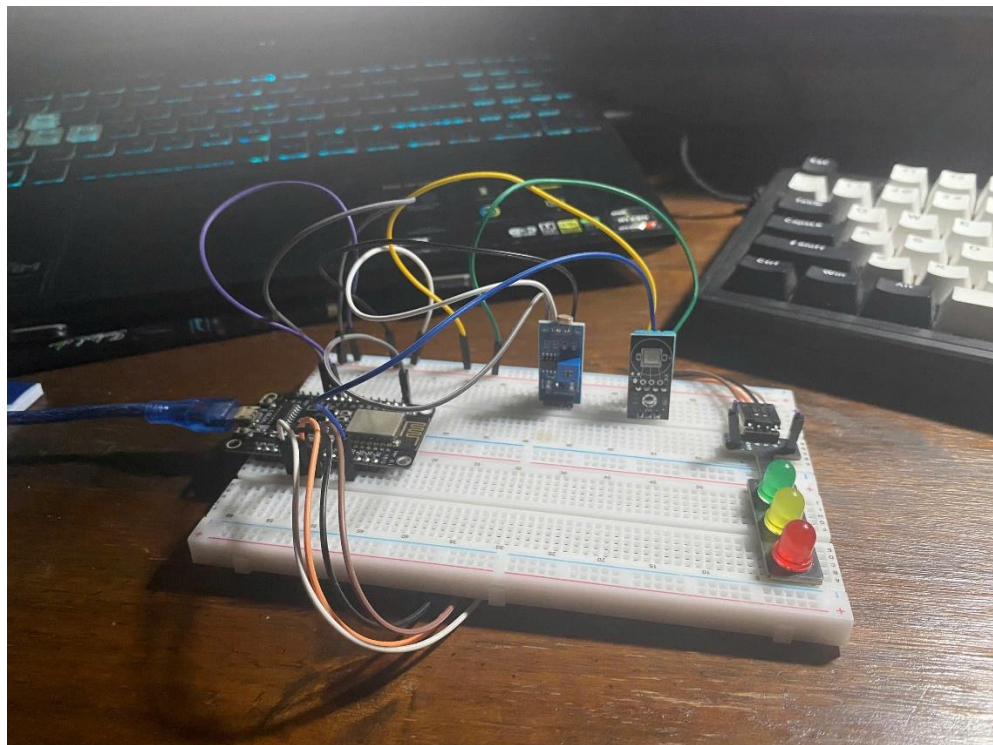
- **Swagger documentation** đầy đủ, tự động sinh tại `/api/docs`.
- **Phân trang, lọc, xác thực đầu vào** toàn cục qua `ValidationPipe` (`whitelist: true, forbidNonWhitelisted: true`).

- **Winston logging** ghi nhận đầy đủ các sự kiện: kết nối MQTT, nhận tin nhắn, lỗi parse, thay đổi trạng thái thiết bị.

4.2. Hình ảnh Demo hệ thống

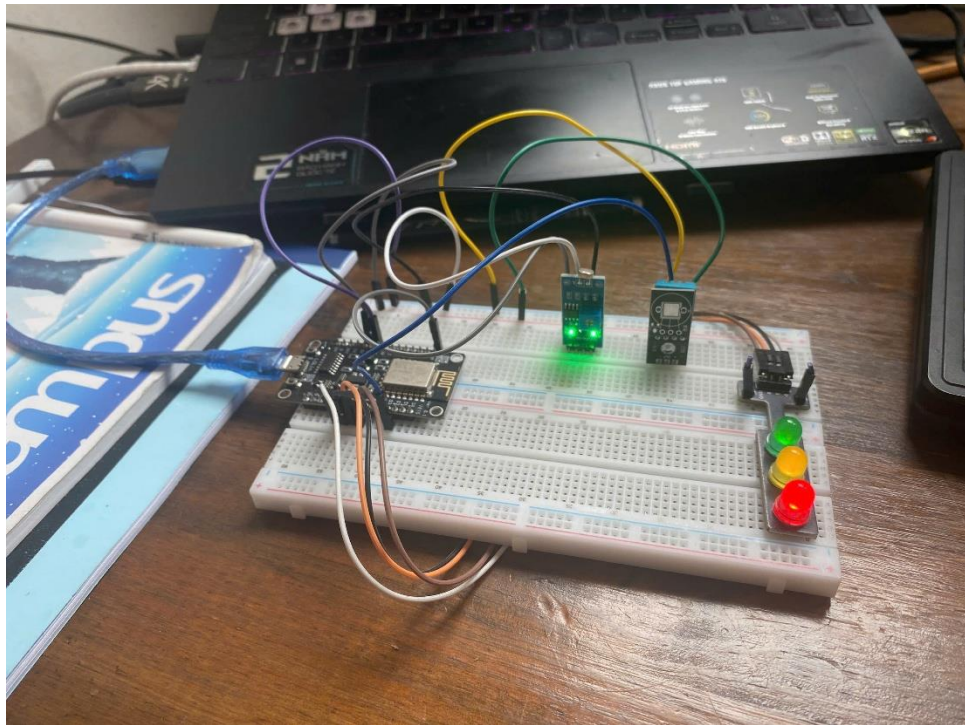
1. Hình ảnh thực tế mô hình phần cứng

Sơ đồ đấu nối tổng thể mô hình IoT trên Breadboard, bao gồm ESP8266 NodeMCU, cảm biến DHT11, LDR và 3 đèn LED:



Hình 4.1: Hình ảnh mô hình phần cứng trên breadboard

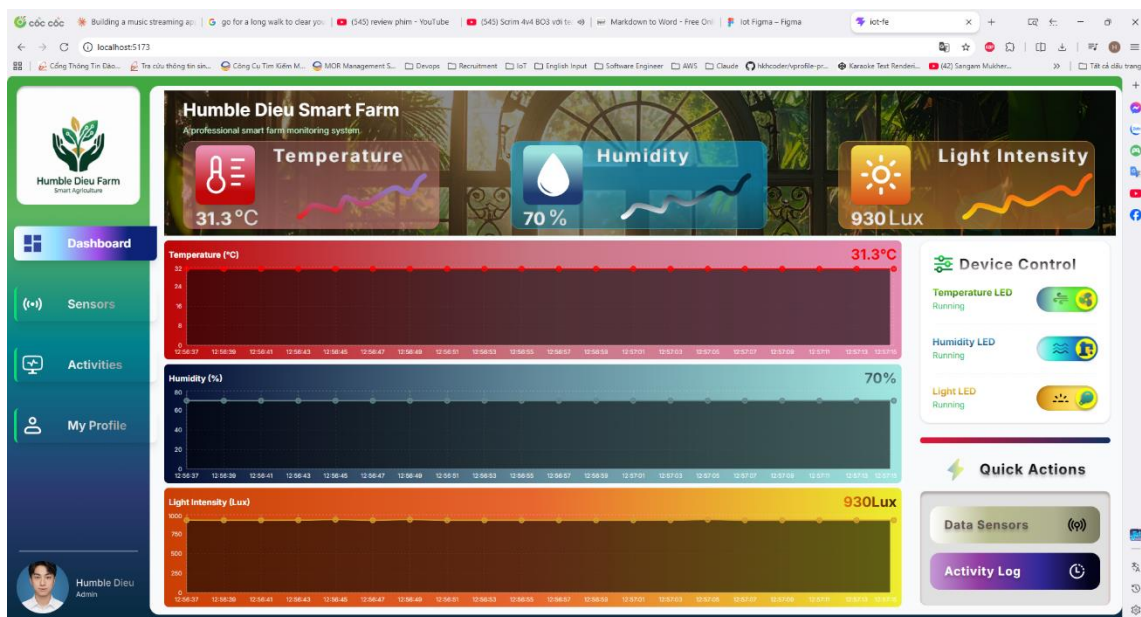
Cận cảnh thiết bị ở trạng thái bật (ON) sau khi nhận lệnh điều khiển từ ứng dụng:



Hình 4.2: Đèn LED ở trạng thái BẬT sau khi nhận lệnh từ Web App

2. Giao diện Web App khi đang vận hành

Giao diện Dashboard hiển thị ba thẻ số liệu Nhiệt độ, Độ ẩm, Ánh sáng cập nhật realtime và ba biểu đồ AreaChart chạy liên tục:



Hình 4.3: Dashboard — thẻ số liệu Hero Banner và biểu đồ realtime

Trang Data Sensors hiển thị lịch sử dữ liệu thu thập được, phân trang rõ ràng, bộ lọc hoạt động chính xác:

SENSOR_ID	TIMESTAMP	SENSOR TYPE	VALUE
#8	2026-04-13 12:57:07	Light Intensity	930 Lux
#8	2026-04-13 12:57:07	Humidity	70 %
#8	2026-04-13 12:57:07	Temperature	31.3 C
#8	2026-04-13 12:57:05	Light Intensity	937 Lux
#8	2026-04-13 12:57:05	Humidity	70 %
#8	2026-04-13 12:57:05	Temperature	31.3 C
#8	2026-04-13 12:57:03	Light Intensity	931 Lux
#8	2026-04-13 12:57:03	Humidity	70 %
#8	2026-04-13 12:57:03	Temperature	31.3 C
#8	2026-04-13 12:57:01	Light Intensity	930 Lux

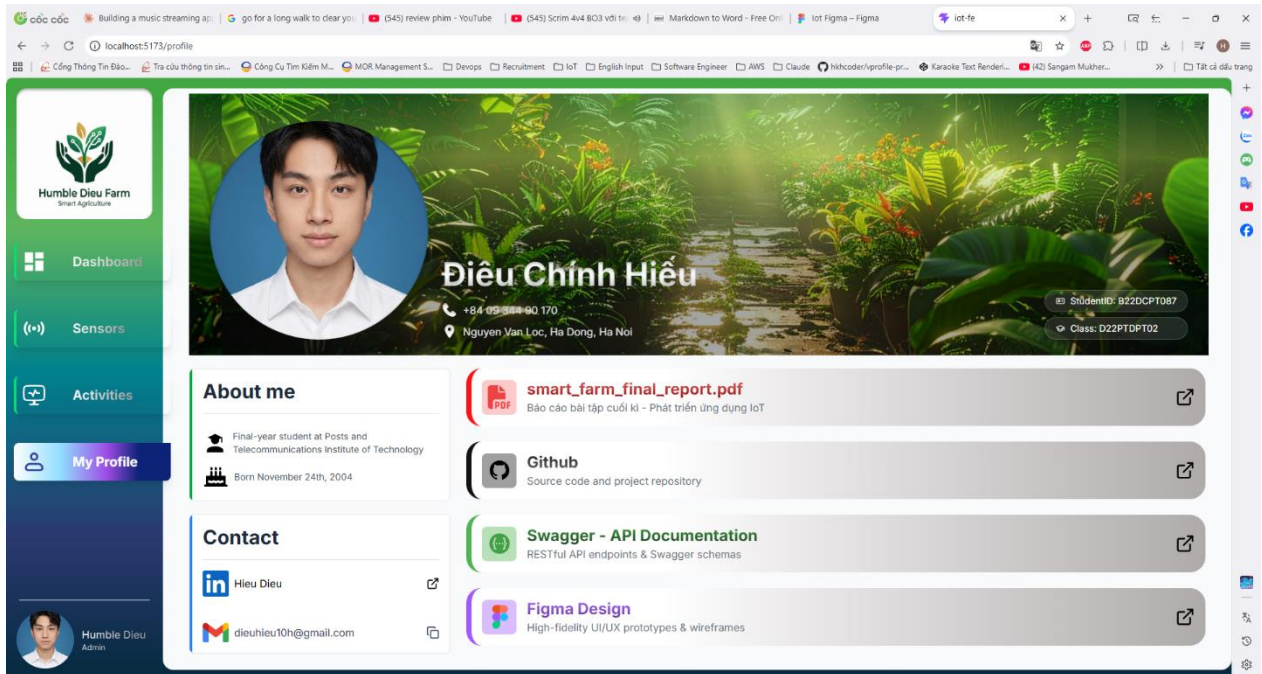
Hình 4.5: Giao diện trang Sensor Data với bảng phân trang

Trang Action Logs ghi nhận đầy đủ nhật ký lệnh điều khiển, badge trạng thái SUCCESS / FAILURE / PROCESSING:

DEVICE_ID	TIMESTAMP	DEVICE NAME	ACTION	ACTION STATUS
#3	2026-04-13 12:55:08	Light LED	TURNED ON	Success
#1	2026-04-13 12:43:28	Light LED	TURNED OFF	Success
#3	2026-04-13 12:43:13	Light LED	TURNED ON	Success
#2	2026-04-13 12:43:11	Humidity LED	TURNED ON	Success
#1	2026-04-13 12:43:10	Temperature LED	TURNED ON	Success
#2	2026-03-29 13:03:31	Temperature LED	TURNED OFF	Success
#2	2026-03-29 13:03:29	Humidity LED	TURNED OFF	Success
#1	2026-03-29 13:03:27	Light LED	TURNED OFF	Success
#2	2026-03-28 12:54:46	Light LED	TURNED ON	Success
#2	2026-03-28 12:54:42	Humidity LED	TURNED ON	Success

Hình 4.6: Giao diện nhật ký hoạt động với badge trạng thái màu sắc

Trang Profile hiển thị thông tin cá nhân sinh viên:



Hình 4.7: Giao diện trang Profile

4.3. KẾT LUẬN

Đề tài "**Hệ thống Giám sát và Điều khiển Nông trại Thông minh – Smart Farm IoT**" đã được triển khai thành công, đáp ứng trọn vẹn các mục tiêu lý thuyết lẫn yêu cầu kỹ thuật thực tế. Thông qua quá trình nghiên cứu và xây dựng, hệ thống đã đạt được những kết quả cốt lõi:

Về kỹ thuật:

- Tích hợp bốn tầng (Edge → Broker → Platform → Application) thành luồng dữ liệu khép kín, độ trễ end-to-end dưới 1 giây cho luồng cảm biến và dưới 2 giây cho luồng điều khiển khứ hồi.
- Thiết kế **5 MQTT topic** tách biệt rõ ràng theo chiều và mục đích, với cơ chế ACK (ON_SUCCESS / OFF_SUCCESS / ON_FAILURE / OFF_FAILURE) đảm bảo xác nhận thực thi ở tầng phần cứng trước khi cập nhật UI.
- Hệ thống **không mất trạng thái** sau sự cố: `currentStatus` trong PostgreSQL là nguồn sự thật duy nhất, ESP8266 luôn đồng bộ từ CSDL khi kết nối lại.

- **NestJS module architecture** phân tách rõ trách nhiệm — thêm loại cảm biến hoặc thiết bị mới chỉ cần thêm entity và đăng ký vào module tương ứng, không cần sửa logic xử lý.

Về chức năng:

- Dashboard realtime với sliding window chart, toggle điều khiển xác nhận ACK, nhật ký kiểm tra đầy đủ vòng đời lệnh, bộ lọc và phân trang linh hoạt cho dữ liệu lịch sử.

Đánh giá chung: Mô hình thử nghiệm đã chứng minh được tính khả thi và độ tin cậy cao. Đây không chỉ là bài toán học thuật mà còn là tiền đề để triển khai mở rộng vào các bài toán thực tiễn như quản lý vi khí hậu nhà màng, cảnh báo môi trường, hoặc tự động hóa thiết bị nông nghiệp diện rộng.

Hướng cải thiện trong tương lai:

#	Tính năng	Mô tả
1	Xác thực & Phân quyền	JWT-based auth, RBAC theo vai trò người dùng
2	Hệ thống cảnh báo	Cảnh báo ngưỡng (nhiệt độ > 35°C → thông báo đẩy)
3	Quy tắc tự động	Điều khiển thiết bị theo lịch trình hoặc điều kiện cảm biến
4	Đa nút IoT	Hỗ trợ nhiều ESP8266, mở rộng phạm vi giám sát
5	Triển khai sản xuất	Tắt <code>synchronize</code> , dùng TypeORM migration, Docker Compose
6	Ứng dụng di động	React Native companion app

TÀI LIỆU THAM KHẢO

1. **NestJS Documentation** — <https://docs.nestjs.com> (Truy cập: 2026)
2. **TypeORM Documentation** — <https://typeorm.io> (Truy cập: 2026)
3. **Đặc tả giao thức MQTT v3.1.1** — Tiêu chuẩn OASIS, 2014
4. **Eclipse Mosquitto MQTT Broker** — <https://mosquitto.org> (Truy cập: 2026)
5. **React 18 Documentation** — <https://react.dev> (Truy cập: 2026)
6. **Recharts Documentation** — <https://recharts.org> (Truy cập: 2026)
7. **Socket.IO Documentation** — <https://socket.io/docs> (Truy cập: 2026)
8. **Tailwind CSS Documentation** — <https://tailwindcss.com/docs> (Truy cập: 2026)
9. **Espressif Systems** — ESP8266 Technical Reference Manual, 2022
10. **Adafruit DHT Sensor Library** — <https://github.com/adafruit/DHT-sensor-library> (Truy cập: 2026)
11. **Nick O'Leary** — PubSubClient MQTT Library for Arduino — <https://github.com/knolleary/pubsubclient> (Truy cập: 2026)
12. **Benoit Blanchon** — ArduinoJson Library — <https://arduinojson.org> (Truy cập: 2026)
13. **Swagger/OpenAPI v3 Specification** — <https://swagger.io/specification> (Truy cập: 2026)
14. **PostgreSQL 16 Documentation** — <https://www.postgresql.org/docs/16> (Truy cập: 2026)
15. **Winston Logger** — <https://github.com/winstonjs/winston> (Truy cập: 2026)